

ADC Los Locos Resumen Técnico y Funcional

Documento de continuidad del proyecto ADC Los Locos preparado para retomar el desarrollo en un nuevo chat. Incluye arquitectura, estructura, flujo funcional, estado actual, refactorización realizada y próximos pasos.

1. Tecnologías y Stack

- Ionic + Angular standalone components
- Firebase Authentication
- Firestore
- Firebase Functions (2nd Gen)
- Angular Guards
- Arquitectura modular por features
- Servicios desacoplados y reutilizables
- Componentes reutilizables para formularios

2. Objetivo funcional de la app

La app ADC Los Locos gestiona socios, invitados, directiva y administración del club. Existen distintos estados de usuario y flujos diferenciados según permisos. Se ha trabajado especialmente el sistema de autenticación, navegación y auditoría.

3. Roles de usuario

- administrador
- directiva
- socio / usuario normal
- invitado pendiente de completar datos

4. Estados de usuario

Enum UserStatus actualmente utilizado:

- ACTIVE
- PENDING_DATA
- PENDING_APPROVAL
- INACTIVE
- REJECTED

5. Flujo completo de invitado

Flujo funcional ya implementado:

1. Directiva/Admin invita usuario
2. Se crea registro parcial Firestore
3. Usuario se registra
4. Login automático Firebase
5. Guard redirige a /complete-profile
6. Usuario completa datos personales
7. Estado pasa a PENDING_APPROVAL
8. Directiva/Admin aprueba usuario
9. Estado pasa a ACTIVE
10. Usuario entra normalmente al home

6. Arquitectura definitiva de navegación

Decisión importante tomada durante el refactor:

- SOLO los guards deciden navegación
- Login únicamente hace navigate("/") tras autenticación
- AuthServiceService NO navega
- CompleteProfile NO navega automáticamente salvo acciones explícitas
- Se eliminaron loops infinitos y race conditions

7. Auth Guard actual

El auth.guard.ts:

- Espera authReady
- Valida login Firebase
- Obtiene currentUserData
- Decide navegación según estado
- Evita loops con validación currentUrl

Redirecciones:

- ACTIVE -> acceso normal
- PENDING_DATA -> /complete-profile
- PENDING_APPROVAL -> /pending-approval
- INACTIVE / REJECTED -> logout + login

8. AuthServiceService

Se refactorizó profundamente:

- initAuthListener()

- login()
- logout()
- reloadUserData()
- currentUser
- currentUserData
- authReady

Importante:

- ya NO navega automáticamente
- limpia currentUserData al cambiar usuario
- evita deadlocks auth

9. Auditoría implementada

Se implementó sistema de auditoría avanzada:

Campos utilizados:

- aprobadoPorUid
- aprobadoPorNombre
- invitedBy
- invitedByName
- creadoPorUid
- creadoPorNombre
- approvedAt
- fechaBaja
- fechaReactivacion

La card de auditoría ya muestra:

- quién realizó acciones
- cuándo
- estados del usuario

10. Firebase Functions

Functions actualmente implementadas:

- createUserByAdmin
- deactivateUser
- reactivateUser
- approveUser

approveUser fue migrada desde frontend a backend para auditoría segura.

11. Formularios reutilizables

Componentes reutilizables ya creados:

- personal-data-form
- membership-form
- user-audit-form

complete-profile utiliza SOLO personal-data-form.

12. Utils reutilizables

Se inició extracción de utilidades comunes a:

src/app/core/Utils/string.utils.ts

Funciones actuales:

- normalizeName()
- capitalize()
- normalizeSpaces()
- isValidEmail()
- formatDNI()

13. Problemas importantes ya solucionados

- loops infinitos de navegación
- auth guard recursivo
- rutas duplicadas
- deadlocks waitForUserData()
- currentUserData stale entre usuarios
- logout sin navegación
- auditoría incompleta
- errores por navegación desde múltiples sitios

14. Estructura aproximada del proyecto

src/app/ ■■■■ core/ ■ ■■■■ services/ ■ ■■■■ guards/ ■ ■■■■ utils/ ■ ■■■■ constants/ ■ ■■■■ features/ ■ ■■■■
auth/ ■ ■ ■■■■ guards/ ■ ■ ■■■■ pages/ ■ ■ ■■■■ services/ ■ ■ ■ ■■■■ users/ ■ ■ ■■■■ components/ ■ ■
■■■■ pages/ ■ ■ ■■■■ models/ ■ ■ ■■■■ services/ ■ ■ ■ ■■■■ shared/ ■ ■■■■ app.routes.ts ■■■■
app.component.ts functions/ ■■■■ src/index.ts

15. Estado actual del proyecto

El sistema auth está MUY avanzado pero aún requiere estabilización final. Actualmente el foco está en:

- navegación definitiva estable
- carga consistente currentUserData

- complete-profile robusto
- evitar loops tras login usuarios pending_data

16. Próximos pasos recomendados

1. Estabilizar completamente auth/navigation
2. Revisar currentUserData async lifecycle
3. Revisar carga datos complete-profile
4. Revisar bindings formulario personal-data-form
5. Añadir resolver/session bootstrap centralizado
6. Mejorar arquitectura reactive state
7. Optimizar guards y lazy loading
8. Continuar funcionalidades club/eventos/socios